Name: _____ **Points:** _____
S-Number: _____ **Date:** _____

Suggested Literature:

- STM32F0x2-Reference-Manual
  - 8 General-purpuse I/Os (GPIO)
  - 18 Geneal-purpose timers (TIM2 and TIM3)
  - 11 Interrupts and events
    - 11.1.3 Interrupt and exception vectors
- ARM® Compiler armasm User Guide
  - Chapter 11.113 __disable_irq intrinsic
  - Chapter 11.113 __enable_irq intrinsic
- Cortex M0 Generic User Guide
  - Exception mask register (page 2-8)

Task 1 (1 pts): Driver Analysis

You are provided with two drivers for this exercise:

- One to get / set `GPIO` pins
- One to access `hardware timers`.

Inspect the files of each driver; and **answer the following questions:**

(a)

- Is the driver interface hardware dependent or independent? Provide an explanation.
- Inspect the `Timer_Context_t` struct; why are some fields declared using the const-keyword while others are not?

Task 2 (5 pts): Interrupt Driven Timer Driver

The provided Timer driver is not comlete yet; we need to adjust it to invoke a callback method when a timer interrupt occurs.

To handle interrupts, we need to add the address of the function representing the interrupt handler to the vector table in the `startup.s` file. The vector table has already been used in the previous exercise; the first entry in the table is the `initial stack pointer`, the second entry is the address of the `Reset_Handler`.**Adjust the vector table so that one entry exists for each entry handler available on our hardware. Unused entries should point to an assembly func that branches to itself.**

Next we implement one interrupt handler function per interrupt in our Timer driver:

```
1  void TIM2_Handler(void);
2  void TIM3_Handler(void);
3  void TIM6_DAC_Handler(void);
4  void TIM7_Handler(void);
5  void TIM14_Handler(void);
6  void TIM15_Handler(void);
7  void TIM16_Handler(void);
8  void TIM17_Handler(void);
```

As the behavior of all timers should be identical, duplicating the code 8 times would be a bad idea. Instead, we **implement the following utility function:**

```
1  static void Timer_ISR(int id);
```

This function should do the following:

- Check if the *TIM_SR_UIF* bit is set in the timer's status register SR.
  - If it is not set the function should do nothing.
- Clear the *TIM_SR_UIF* flag.
- Increment the timer's tick counter.
- Call the callback function (if it is not *NULL*).

Now that we implemented the interrupt handler functions, we need to **add them to the vector table** in the Startup.s file. **Make sure to enable interrupts** using the *__enable_irq* intrinsic in your main function.

(a) ☐ **Implement the interrupt handler functions** so that they just call the *Timer_ISR* function with the correct id.

Task 3 (8 pts): Event Queue

Create a new module named `EventQueue`. This module provides functions to pass information about events to the state machine which we will implement in a later task.

The module should provide the following interface:

```
/******************************************************************************
Provides information about an event.
******************************************************************************/
typedef struct Event_s {
    int32_t EventType; // The type of the event (e.g. Button pressed)
    int32_t X; // Additional information (meaning depends on event type).
    int32_t Y; // Additional information (meaning depends on event type).
    int32_t Z; // Additional information (meaning depends on event type).
} Event_t;

/******************************************************************************
Initializes the event queue module.
Must be called exactly once before any other functions.
returns: Zero on success, an error code otherwise.
******************************************************************************/
int EventQueue_Initialize(void);

/******************************************************************************
Adds an event to the event queue.
event: A pointer to the event to add.
returns: Zero on success, an error code otherwise.
0: Success
1: Event queue is full; event not added to the queue.
******************************************************************************/
int EventQueue_Put(Event_t const * event);

/******************************************************************************
Gets an event from the event queue.
event: A pointer to an event.
returns: Zero on success, an error code otherwise.
0: Success
1: Event queue is empty
******************************************************************************/
int EventQueue_Get(Event_t * event);

/******************************************************************************
Clears the event queue.
******************************************************************************/
void EventQueue_Clear(void);
```

The module will be used to pass events detected by interrupt handlers to the application state machine; make sure to protect shared memory via a critical section/by disabling and re-enabling interrupts. Make sure the maximum number of events which can be stored in the event queue is configurable via a compile time constant (#define).

(a) ☐ Create the module named `EventQueue`.

Task 4 (6 pts): Key Module

The Key module is used to periodically check the state of the following keys:

- User0
- User1
- Wake-Up

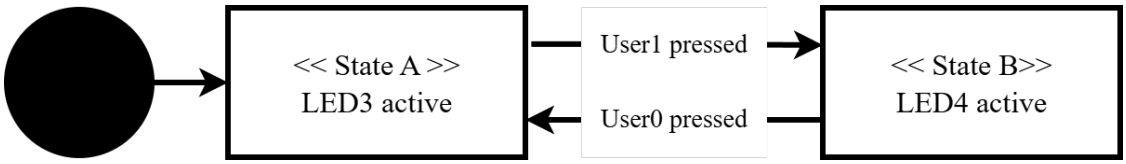
The module uses the `Timer` and `GPIO` driver to periodically (every `50 ms`) check the state of each key. It manages a counter for each key which is incremented while the key is pressed and reset to zero when the key is released. Before the counter is reset to `0`, the counter value is used to determine whether a key press event occurred. If so, an event is added to the event queue (implemented in a previous task). An event should only be generated if the key has been pressed for `100 ms` or more and less than `1000 ms`.

EventType	**X (Key)**	**Y (Not used)**	**Z (Not used)**
1 ... KeyPressed	0 ...User 0 1 ...User 1	**0**	0

(a) ☐ Implement the Key Module to check the states of each key.

Task 5 (4 pts): State Machine

We will use a state machine to model our application. In this task we start with a simple state machine to test the modules implemented in the previous tasks:



Implement a new module named StateMachine which provides a single function StateMachine_Run:

```
1  typedef void (*StateTransitionFunction_t)(void);
2  typedef struct State_s const * (*StateFunction_t)(void);
3  typedef struct State_s {
4      StateTransitionFunction_t Enter;
5      StateFunction_t Process;
6      StateTransitionFunction_t Exit;
7  } State_t;
8  void StateMachine_Run(State_t const * state);
```

(a) ☐ **Implement the state machine (create a separate file for each state) using the already available modules and drivers. Perform the necessary initialization in the main function (before entering the infinite loop) and call StateMachine_Run to start the state machine (note that this function will never return).**

UAS Upper Austria - Campus Hagenberg